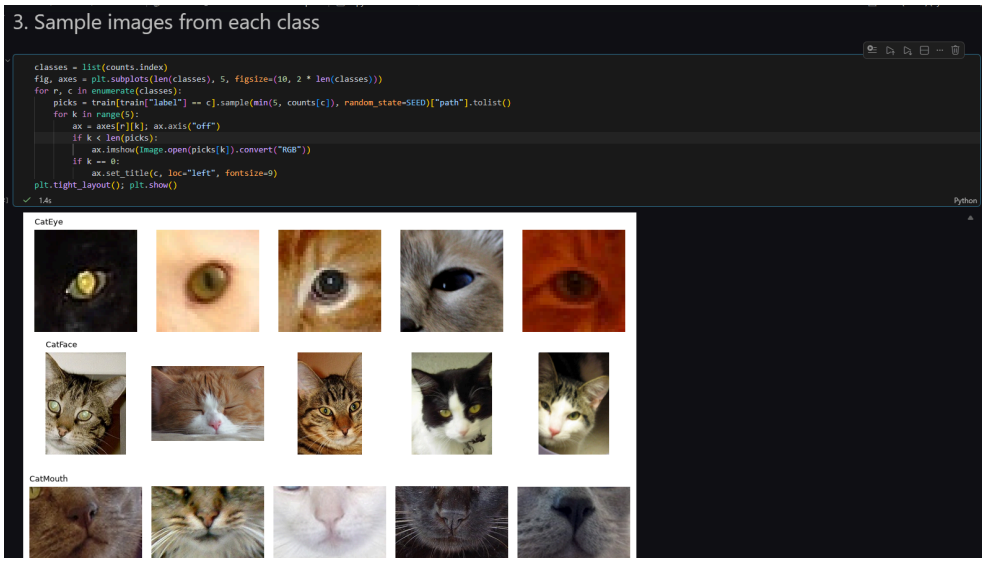
Facial Feature Classification Challenge

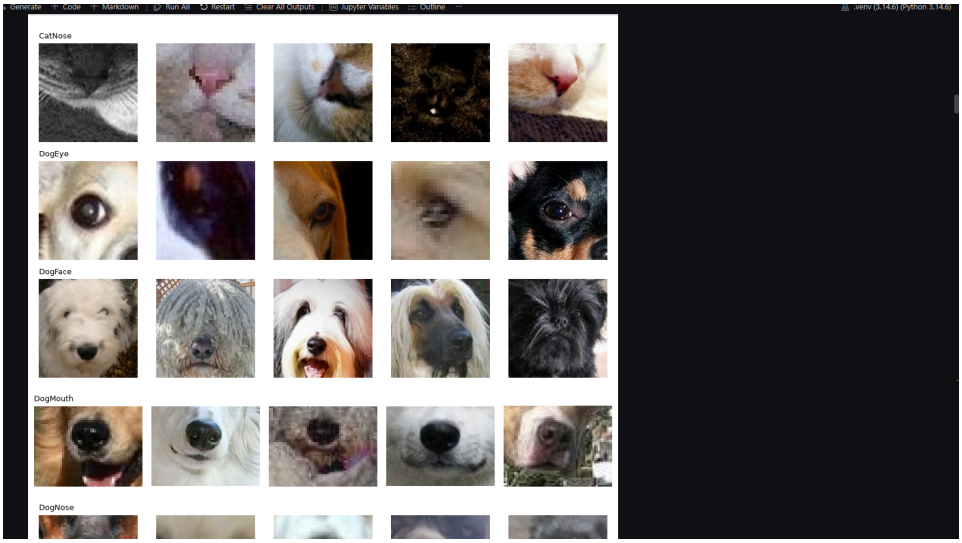
Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Task 1: Dataset Exploration

Dataset and classes

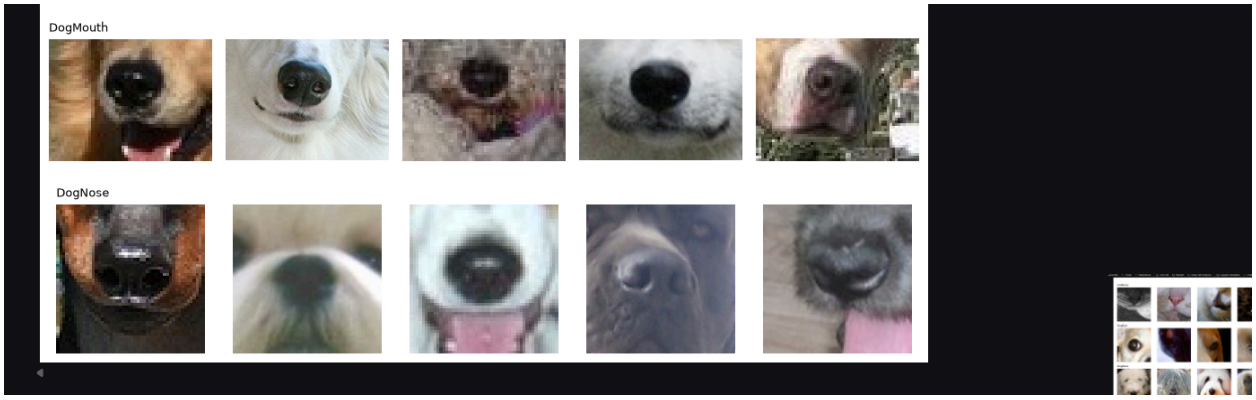
The images in the dataset are stored as JPEG files. There are 63,660 labeled training images and 29,443 unlabeled test images. The images show one of the eight facial features of a cat or a dog. The eight classes are CatEye, CatFace, CatMouth, CatNose, DogEye, DogFace, DogMouth and DogNose. Every training image has a corresponding label and every label has an image file, all of which are valid and can be read.





Facial Feature Classification Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye



Class balance

The classes are extremely imbalanced . The CatEye class has 13,884 images (21.81%) whereas the three classes that represent dogs have only 5,790 images each (9.10%) on average. Thus, the largest class accounts for more than twice as many images as the smallest class. Thus, a simple model that always predicts CatEye would already achieve majority-class accuracy of 21.8%. Because of the imbalanced data, stratified split was used in Section 2, and also report per-class accuracy in addition to the overall accuracy for all methods.

Class	Images	Share (%)
CatEye	13,884	21.81
CatFace	6,942	10.90
CatMouth	6,942	10.90
CatNose	6,942	10.90
DogEye	11,580	18.19
DogFace	5,790	9.10
DogMouth	5,790	9.10
DogNose	5,790	9.10

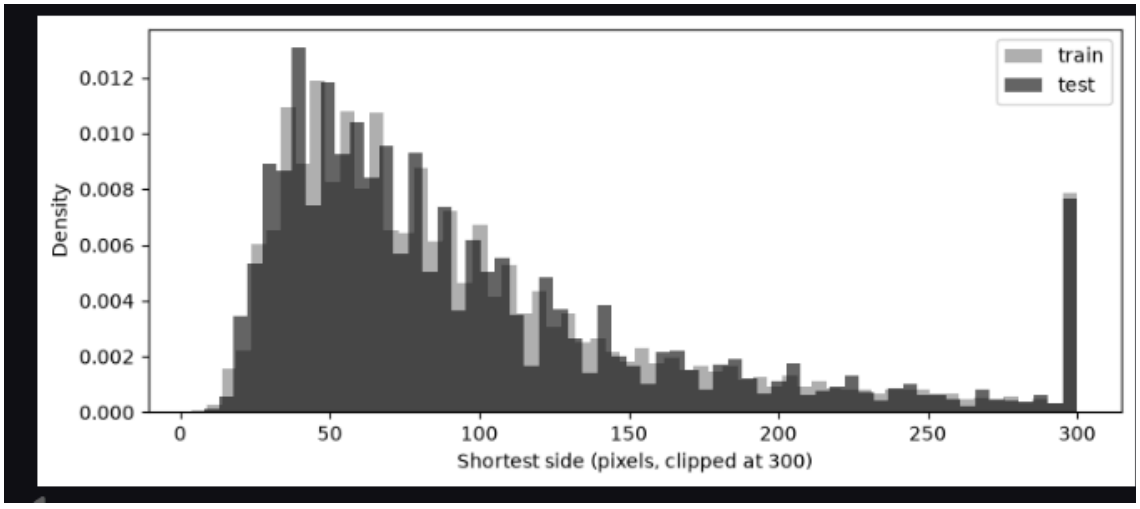
Facial Feature Classification Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye



Image size

The size of images within the training data ranged from 4x4 pixels up to 2219 x 2219 pixels with 7,861 sizes used in total. The size of the shortest side of the images was found to have a median of 78px. 37.2% of all images were of size less than 64x64 pixels and 69% of all images were square in size. The size of the images was found to vary from class to class with the images of CatFace having a median size of 182 x 215 pixels. The images of DogEye and DogNose had a size of 62 x 62 pixels on average. A simple model that used the width and/or height of the images as input was able to achieve a classification accuracy of 45.8% with 5-fold cross-validation. This is far greater than the majority-class accuracy of 21.8%. The size of the images within the test set followed a similar distribution (median of the shortest side of 77px, 40.4% of less than 64x64 pixels) to that within the training set.

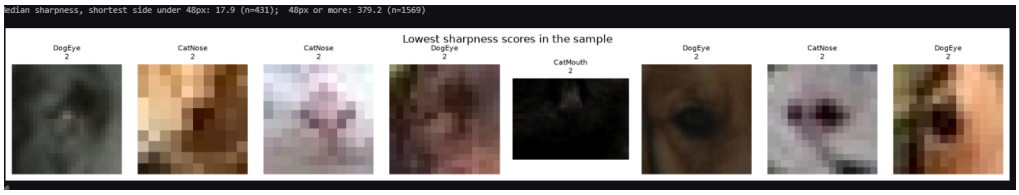


Facial Feature Classification Challenge

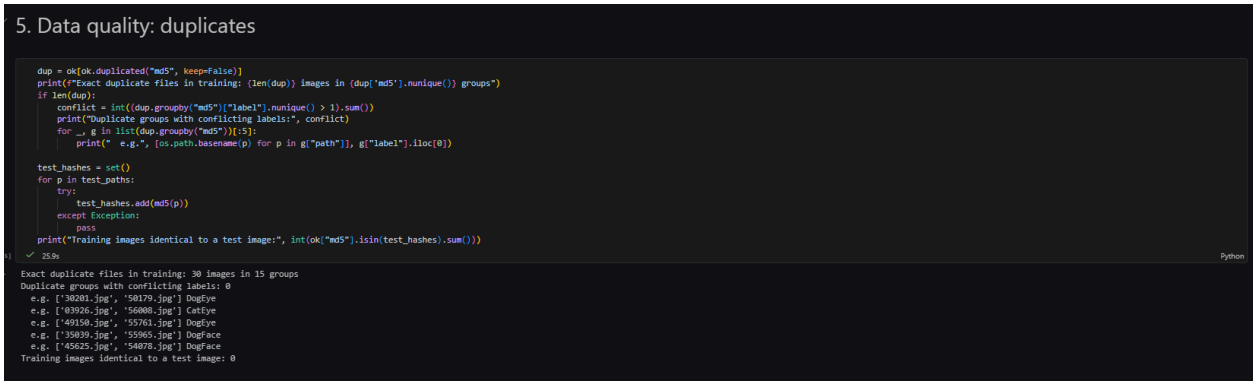
Student name: Matty Luriz ID: 23206450 Username: Outer Skye

Image quality and duplicates

The small images were often blurry. The sharpness was measured as the variance of a Laplacian filter on a 128 x 128 grayscale copy of the image. Such a sharpness value was determined for a random sample of 2,000 training images. The median value of the sharpness was 17.9 for the images with the shortest side of less than 48 px (n=431) and 379.2 for the images with a shortest side greater than 48 px (n=1,569). Low values denote blurry images.



This dataset contains 30 training images. 15 of these images are in exact duplicate form and so form 15 groups of duplicates with identical MD5 hashes. None of these groups contain conflicting labels. No training images are duplicates of test images. There are only a small number of duplicates in total so it is not sensible to put a copy of a duplicate image in the validation set and its twin in the training set set as this would lead to overinflation of the accuracy on the validation set. So all of the duplicates are kept together when the data is split.



Task 2: Data Preprocessing and Pipeline Preparation

Resizing and normalisation

Class labels are encoded as integers from 0 to 7 in alphabetical order, so it is easy to write out a classification prediction as a class name. After encoding the labels of the images they were split into 80% training set and 20% validation set. Stratified split was used with duplicates in the original images grouped together. Split is done using scikit-learn's StratifiedGroupKFold class with a random state fixed to a constant value across calls, i.e. hashing of files, so the same images are used in subsequent runs. The split has 50,928 images in the training set and 12,732

Facial Feature Classification Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye
images in the validation set. The largest difference in class share between the training set and the validation set is 0.01 percentage points. The split is saved to a file. It was also verified that there are no identical files and no duplicate image paths in the training and validation sets in.

Data set	Images	Share of labelled data (%)
Training	50,928	80.0
Validation	12,732	20.0
Test (unlabelled)	29,443	not applicable

2. Train/validation split

```
n_splits = round(1 / VAL_FRACTION)
sgkf = StratifiedGroupKFold(n_splits=n_splits, shuffle=True, random_state=SEED)
train_idx, val_idx = next(sgkf.split(data, data["y"], groups=data["md5"]))
train_df = data.iloc[train_idx].reset_index(drop=True)
val_df = data.iloc[val_idx].reset_index(drop=True)

share = pd.DataFrame({
    "train_n": train_df["label"].value_counts().sort_index(),
    "val_n": val_df["label"].value_counts().sort_index()})
share["train_%"] = (100 * share["train_n"] / share["train_n"].sum()).round(2)
share["val_%"] = (100 * share["val_n"] / share["val_n"].sum()).round(2)
display(share)
print(f"Training: {len(train_df)} images ({100 * len(train_df) / len(data):.1f}%)   "
      f"Validation: {len(val_df)} images ({100 * len(val_df) / len(data):.1f}%)")
print(f"Largest class-share difference between the sets: {(share['train_%'] - share['val_%']).abs().max():.2f} percentage points")

# Save the split so every later task and every rerun uses exactly the same images
split = pd.concat([train_df.assign(split="train"), val_df.assign(split="val")])[["path", "label", "split"]]
split.to_csv(SPLIT_FILE, index=False)
print("Saved", SPLIT_FILE)
```

23.6s Python

label	train_n	val_n	train_%	val_%
CatEye	11107	2777	21.81	21.81
CatFace	5554	1388	10.91	10.90
CatMouth	5554	1388	10.91	10.90
CatNose	5553	1389	10.90	10.91
DogEye	9264	2316	18.19	18.19
DogFace	4632	1158	9.10	9.10
DogMouth	4632	1158	9.10	9.10
DogNose	4632	1158	9.10	9.10

```
Training: 50928 images (80.0%)   Validation: 12732 images (20.0%)
Largest class-share difference between the sets: 0.01 percentage points
Saved split.csv
```

3. Leakage checks

```
dup_groups = data[data.duplicated("md5", keep=False)]["md5"].nunique()
shared_hashes = set(train_df["md5"]) & set(val_df["md5"])
shared_paths = set(train_df["path"]) & set(val_df["path"])
print(f"Duplicate groups in the labelled data: {dup_groups}")
print("Identical files in both training and validation:", len(shared_hashes))
print("Image paths in both training and validation:", len(shared_paths))
assert not shared_hashes and not shared_paths, "leakage between training and validation"

val_dup_in_train = val_df["md5"].isin(set(train_df["md5"])).sum()
print("Validation images with an exact copy in training:", int(val_dup_in_train))
print("Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.")
```

0.1s Python

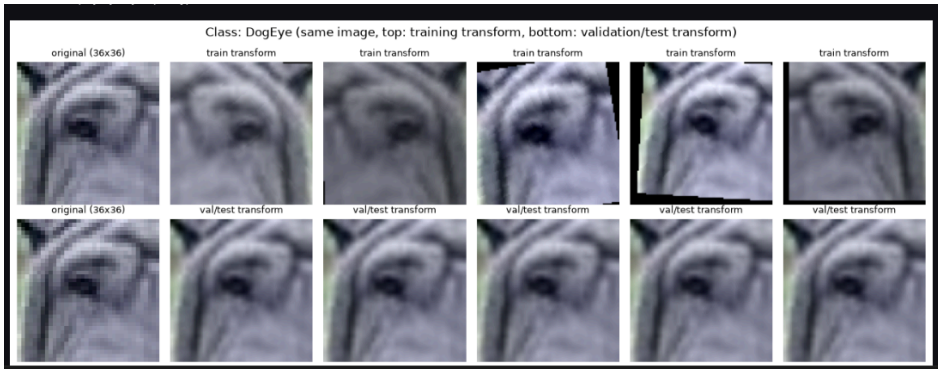
```
Duplicate groups in the labelled data: 15
Identical files in both training and validation: 0
Image paths in both training and validation: 0
Validation images with an exact copy in training: 0
Normalisation constants are fixed ImageNet values, so no statistic is learned from validation or test images.
```

Augmentation

To augment the training set a number of mild transforms have been applied. These include a horizontal flip with a 0.5 probability, random rotation up to 10 degrees, random shift up to 5%, and random scale between 0.9 and 1.1. Additionally there are random changes to the brightness, contrast, and color saturation of the images. This is safe as the shape of the faces (mouth, nose, etc.) are roughly left-right symmetrical. Vertical flips and large rotations have not been used as the images appear to be upright. The validation and test sets have had one transform applied to them in the same way as for the training set but this simply resizes and

Facial Feature Classification Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye
then normalises the images. An example of the same image going through a training and a validation/test transform is shown in.



Data pipeline

The dataset class simply reads the images from disk, converts them to RGB and applies the appropriate transform. The data loaders are set up to use a batch size of 64, shuffle the training set using a fixed seed generator, and use zero workers (because, in Windows notebooks, multiprocessing is extremely unreliable). The test images are loaded in exactly the same order as the sample submission file, and check that every id in the file has a corresponding test image. In the finished pipeline, batches are of shape (64, 3, 96, 96) with values having approximately zero mean and unit standard deviation after normalization.

```
train batch: (64, 3, 96, 96)  dtype torch.float32  mean -0.19  std 1.23  min -2.12  max +2.64
val   batch: (64, 3, 96, 96)  dtype torch.float32  mean -0.10  std 1.11  min -2.12  max +2.64
test  batch: (64, 3, 96, 96)  dtype torch.float32  mean -0.06  std 1.10  min -2.12  max +2.64
```

Task 3: Model Selection and Pipeline Development

Model choice

For this experiment ResNet-18 was chosen, pre-trained on ImageNet (He et al., 2016) for transfer learning. ResNet-18 is a residual CNN that has approximately 11.2M parameters and can be trained relatively fast on a laptop CPU (Intel Core i7-10750H, CPU only). Importantly, the edges, textures and shapes that the network has learned from images in ImageNet are expected to transfer to images of cat and dog faces. Importantly, other models such as the Vision Transformer ViT-B/16 (Dosovitskiy et al., 2021) that was used in this experiment has approximately 86M parameters and is much slower when trained on a CPU and expects input of size 224×224. A custom CNN would also need to be trained from scratch and would in all likelihood require more epochs and more data to converge than the ResNet that were pre-trained.

Facial Feature Classification Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

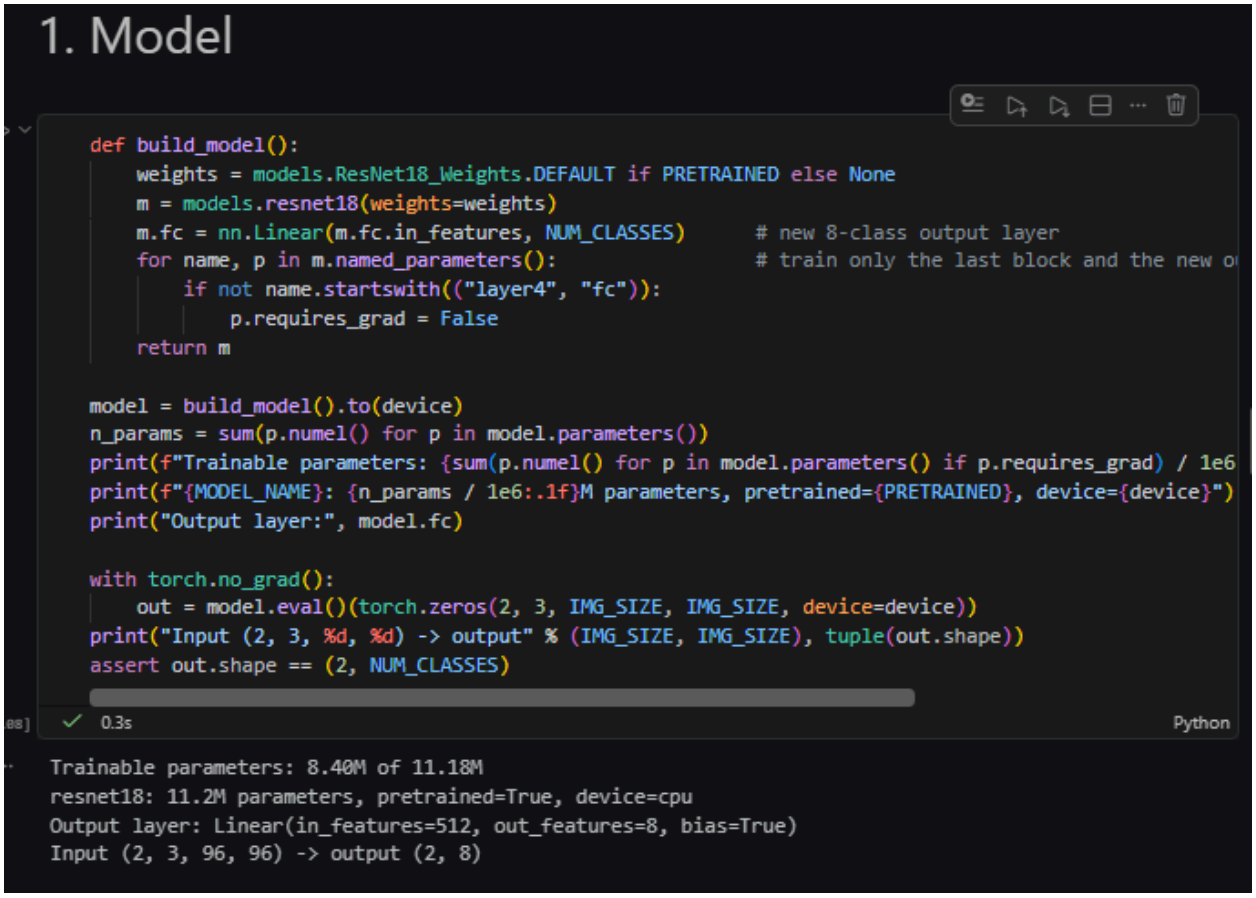
Model Configuration

The input size of the network is 3 x 96 x 96 from the preprocessing stage. The last fully connected layer is replaced with a linear layer of 8 output units. All the other layers use ReLU as the activation function, and no activation function is applied on the output layer in order to get raw scores (logits) of the network output that will be later softened by the softmax function within the loss function. Cross-entropy loss is suitable for single-label classification with 8 output classes. AdamW (Loshchilov & Hutter, 2019) is used as the optimizer with a low learning rate of 3e-4 and a high weight decay of 1e-4 in order to fine-tune the pre-trained weights slowly without ‘breaking’ them. A fixed batch size of 32 is used in order to get stable estimates of the gradients, and the number of epochs is set to 5 in order to fit the laptop CPU.
In the baseline the last residual block (layer4) and the newly added output layer are trained (8.40M out of 11.18M parameters total). Early layers in the network are fixed and detect basic edges and textures that can be transferred to the task at hand, and thus only a portion of the backpropagated gradients need to be computed in each of the 5 epochs. That ran in about 13 minutes on the laptop’s CPU.

Setting	**Value**
Model	ResNet-18 with ImageNet pretrained weights
Input	3 x 96 x 96, ImageNet normalisation
Output layer	Linear layer, 512 inputs to 8 outputs (logits)
Trainable parts	layer4 and the output layer (8.40M of 11.18M parameters)
Loss	Cross-entropy
Optimiser	AdamW, learning rate 3e-4, weight decay 1e-4
Batch size	32
Epochs	5
Model selection	Weights from the epoch with the best validation accuracy
Hardware	Intel Core i7-10750H, CPU only

Facial Feature Classification Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye



Training and prediction pipeline

We use three functions to build a pipeline. The first function runs one pass over the data in a given data loader (either training or evaluating), returning loss and accuracy. The second function trains the model for a number of epochs, saving out the training and validation loss and accuracy after each epoch. The weights for the model with the best validation accuracy are also saved out. This function can also take in a learning-rate scheduler and early-stopping patience parameters, which are used in Section 4. The third function loads in the weights for the best validation accuracy and uses the model to predict a class for each test image one by one as they are listed in the sample submission file, and the resulting file is checked against sample_submission.csv (column names, number of rows, image IDs in order, class names) before being saved as a file that can be submitted to Kaggle. Before running the full script, we do a speed check by running a few batches and timing how long it takes to complete an epoch, and also include a debug option that runs the whole script on a very small subset of the data first.

Facial Feature Classification Challenge

Student name: Matty Luriz ID: 23206450 Username: Outer Skye

2. Data loaders for training

```
def make_loader(ds, shuffle, seed=SEED):
    if DEBUG_N:
        ds = Subset(ds, range(min(DEBUG_N, len(ds))))
    gen = torch.Generator().manual_seed(seed) if shuffle else None
    return DataLoader(ds, batch_size=TRAIN_BATCH_SIZE, shuffle=shuffle, generator=gen,
                      num_workers=NUM_WORKERS, pin_memory=(device == "cuda"))

fit_train_loader = make_loader(train_ds, shuffle=True)
fit_val_loader = make_loader(val_ds, shuffle=False)
fit_test_loader = make_loader(test_ds, shuffle=False)
print(f"batches per epoch: train {len(fit_train_loader)}, val {len(fit_val_loader)}, test {len(fit
      f"  (batch size {TRAIN_BATCH_SIZE})" + ("  [DEBUG subset]" if DEBUG_N else ""))
```

0.0s Python

```
batches per epoch: train 1592, val 398, test 921  (batch size 32)
```

4. Speed check

```
# Times a few training batches on a throwaway copy of the model to estimate the length of one epoch
probe = copy.deepcopy(model).train()
probe_opt = torch.optim.AdamW(probe.parameters(), lr=LEARNING_RATE)
probe_scaler = torch.amp.GradScaler(device, enabled=use_amp)
n_probe, t0 = 0, None
for i, (xb, yb) in enumerate(fit_train_loader):
    if i == 2:                  # skip the first batches (warm-up)
        if device == "cuda":
            torch.cuda.synchronize()
        t0 = time.time()
    xb, yb = xb.to(device), yb.to(device)
    with torch.autocast(device_type=device, enabled=use_amp):
        loss = nn.functional.cross_entropy(probe(xb), yb)
    probe_opt.zero_grad(set_to_none=True)
    probe_scaler.scale(loss).backward(); probe_scaler.step(probe_opt); probe_scaler.update()
    if i >= 2:
        n_probe += 1
    if i == 11 or i == len(fit_train_loader) - 1:
        break
if device == "cuda":
    torch.cuda.synchronize()
if n_probe:
    per_batch = (time.time() - t0) / n_probe
    print(f"{per_batch:.2f}s per batch -> about {per_batch * len(fit_train_loader) / 60:.1f} min per training epoch "
          f"(plus validation), about {per_batch * len(fit_train_loader) * EPOCHS / 60:.0f} min for {EPOCHS} epochs")
if device == "cuda":
    print(f"peak GPU memory: {torch.cuda.max_memory_allocated() / 2**30:.2f} GiB of {torch.cuda.get_device_properties(0).total_memory / 2**30:.1f} GiB")
del probe, probe_opt
```

52s Python

```
0.42s per batch -> about 11.2 min per training epoch (plus validation), about 56 min for 5 epochs
```